BURRA SEC

Zaiffer Protocol Security Review

Reviewed by: Goran Vladika, 0x37, santipu

19th November - 26th November, 2025

Zaiffer Protocol Security Review Report

Burra Security

December 29, 2025

Introduction

A time-boxed security review of the **Zaiffer** protocol was done by **Burra Security** team, focusing on the security aspects of the smart contracts.

Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any vulnerabilities. Subsequent security reviews, bug bounty programs, and on-chain monitoring are recommended.

About Burra Security

Burra Sec offers security auditing and advisory services with a special focus on cross-chain and interoperability protocols and their integrations.

About Zaiffer

Zaiffer is a confidential token system built on Zama's FHEVM that implements ERC7984 tokens with encrypted balances and transfer amounts, enabling private transactions while maintaining full regulatory compliance through sanctions checking and regulator access controls. The system uses a modular

architecture with isolated wrapper contracts for fund custody, centralized admin configuration for fee and policy management, and supports advanced features like batch transfers and confidential token swaps through external DEXs.

Severity classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact - The technical, economic, and reputation damage from a successful attack

Likelihood - The chance that a particular vulnerability gets discovered and exploited

Severity - The overall criticality of the risk

Informational - Findings in this category are recommended changes for improving the structure, usability, and overall effectiveness of the system.

Security Assessment Summary

***review commit hash* - 0922f61f27ae9d02cd74993de50a7b8c3023ff94**

Scope

The following smart contracts were in the scope of the audit:

- contracts/admin/AdminProvider.sol
- contracts/admin/FeeManager.sol
- contracts/admin/SanctionsList.sol
- contracts/batcher/ERC7984TransferBatcher.sol
- contracts/factory/DeploymentCoordinator.sol
- contracts/factory/RegulatedERC7984UpgradeableFactory.sol
- contracts/factory/WrapperFactory.sol
- contracts/swap/swap_v0.sol

- contracts/token/RegulatedERC7984Upgradeable.sol
- contracts/wrapper/WrapperUpgradeable.sol

Findings Summary

ID	Title	Severity	Status
H-1	Fee-on-transfer tokens cause permanent fund loss in SwapV0	High	Resolved
H-2	Loss of funds when wrapping tokens with large total supply	High	Resolved
H-3	User's underlying asset may be stolen if the token's transfer has a hook	High	Resolved
M-1	Sanctioned recipient addresses can cause funds to be unrecoverable	Medium	Resolved
M-2	Loss of fees due to rounding down in `FeeManager`	Medium	Resolved
M-3	Creation of `ERC7984` and `WrapperUpgradeable` for non-existent tokens	Medium	Resolved
M-4	Locked funds when unwraping if user uses Account Abstraction	Medium	Resolved
M-5	Swap refunds triggered by low input in `getAmountsOut`	Medium	Resolved
M-6	Arbitrary `routerAddress` allows fund theft and fee bypass	Medium	Resolved
M-7	Sanctioned entities can still operate token actions as approved operators	Medium	Resolved
M-8	Permissionless finalizeUnwrap can allow anyone to trigger a forced refund	Medium	Resolved
M-9	Custom token route enables wrap/unwrap fee circumvention	Medium	Resolved
M-10	Missing one interface to update FHE's coprocessor	Medium	Acknowledged
M-11	Users may lose funds if transfer fails	Medium	Resolved
L-1	Removal of leading zeroes when deriving name/symbol from address	Low	Resolved
L-2	Contracts fail to reflect updated `adminProvider` from `DeploymentCoordinator`	Low	Resolved

ID	Title	Severity	Status
L-3	Invalid router address causes permanent fund loss	Low	Resolved
L-4	Zero address swap recipient causes token-cToken parity break	Low	Resolved
L-5	Transfer batcher doesn't allow operators	Low	Resolved
L-6	Potential stealing funds from `SwapV0`	Low	Resolved
L-7	Callback return value ignored	Low	Resolved
I-1	Inaccurate `actualFeeReceived` due to unaccounted `transferDust`	Info	Resolved
I-2	cETH is not supported in swap_v0	Info	Resolved
I-3	Batcher retry mechanism leaks info about failed transactions	Info	Acknowledged
I-4	Minor code improvements	Info	Resolved

Detailed Findings

[H-1] Fee-on-transfer tokens cause permanent fund loss in SwapV0

Target

- swap_v0.sol#L245

Severity

- Impact: High
- Likelihood: High

Description

SwapV0 facilitates atomic swaps between confidential tokens by unwrapping them to regular tokens, swapping via Uniswap V2, and wrapping the output back to confidential form. The contract assumes the amount of tokens it receives matches the amount sent by the wrapper, but this assumption breaks

with fee-on-transfer (FOT) tokens. Worth mentioning, protocol explicitly supports FOT tokens and includes logic to handle FOT tokens in the wrapper.

When finalizing an unwrap in `WrapperUpgradeable._finalizeSuccessfulUnwrap()`, the wrapper transfers tokens to SwapV0:

```
// Transfer principal to receiver
bool unwrapSuccess = _transferUnderlying($._originalToken, receiver.to,
    unwrapAmount);
```

For a 2% transfer tax token, if unwrapAmount = 1000, SwapV0 only receives 980 tokens due to the transfer tax. However, the wrapper proceeds to call SwapV0 with the original amount:

```
if (receiver.to.code.length > 0) {
    IWrapperReceiver(receiver.to).onUnwrapFinalizedReceived(
        msg.sender,
        unwrapAmount,  // Still 1000, not aware it's 980!
        requestId,
        receiver.callbackData
    );
}
```

SwapV0 then attempts to swap tokens it doesn't have. In `onUnwrapFinalizedReceived()`:

```
IERC20(path[0]).forceApprove(routerAddress, amountIn);  // Approves
    1000
try IUniswapV2Router02(routerAddress).swapExactTokensForTokens(
    amountIn,  // Attempts to swap 1000, but only has 980
    amountOutMin,
    path,
    address(this),
    deadline
) returns (uint256[] memory amounts) {
    _handleUniswapSuccess(path, amounts, to, unwrapRequestId);
} catch {
    returnVal = _refundUser(...);  // Refund path
}
```

The router's attempt to pull 1000 tokens fails since SwapV0 only holds 980, triggering the catch block. The refund mechanism in _refundUser() has the same flaw:

```
function _refundUser(...) internal returns (bool) {
    // ...
    wrapperIn.wrap(to, amountIn);  // Tries to wrap 1000, only has 980!
    // ...
}
```

The critical issue is this refund call occurs outside any try-catch:

- The wrapper's transferFrom(SwapV0, wrapper, 1000) fails with insufficient balance

- The entire finalizeUnwrap transaction reverts
- The callback data with incorrect amount is immutably stored in `_receivers[requestId]`
- Every retry hits the same revert, leaving funds permanently stuck

As a consequence, user's cTokens are burned, while the underlying tokens are permanently stuck.

Proof of Concept:

```
it.only("should work with FOT tokens", async function () {
  // create feeToken and cFOT
  const feeToken = await deployFeeOnTransferToken(signers, 100);
  await feeToken.mint(signers.alice, 100_000_000_000);
  let cFOT: RegulatedERC7984Upgradeable;
  let wrapperFOT: Wrapper;
  ({ cToken: cFOT, wrapper: wrapperFOT } = await deployConfidentialToken(coordinator, feeToken, signers.alice));
  await wrapERC20(coordinator, feeToken, BigInt(100_000), signers.alice.address, signers.alice);

  // add liquidity with feeToken
  pair = await addLiquidity(signers.alice, router, feeToken, 100_000_000, tokenB, 100_000_000);

  const transferAmount = BigInt(1_000);
  const encryptedTransferAmount = await fhevm
    .createEncryptedInput(await cFOT.getAddress(), signers.alice.address)
    .add64(transferAmount)
    .encrypt();

  const path = [await feeToken.getAddress(), await tokenB.getAddress()];

  const abiCoder = new ethers.AbiCoder();
  const callbackData = abiCoder.encode(
    ["address", "uint256", "address[]", "uint256", "address"],
    [await router.getAddress(), 0, path, Math.floor(Date.now() / 1000) + 6000, signers.alice.address],
  );
  const data = abiCoder.encode(["address", "bytes"], [await swapV0.getAddress(), callbackData]);

  // phase 1
  console.log("Phase1 start: confidentialTransferAndCall");
  const transferTx = await cFOT
    .connect(signers.alice)
    [
      "confidentialTransferAndCall(address,bytes32,bytes,bytes)"
    ](await wrapperFOT.getAddress(), encryptedTransferAmount.handles[0], encryptedTransferAmount.inputProof, data);
  const transferReceipt = await transferTx.wait();
```

```
    console.log("Phase1 complete");

    // phase 2
    console.log("Phase2 start: finalizeUnwrap");
    await getSwapFinalizedReceipt(transferReceipt, wrapperFOT, signers.
        bob);
    console.log("Phase2 complete");
  });
```

Run it:

```
  Uniswap V2 Swap Test
Phase1 start: confidentialTransferAndCall
Phase1 complete
Phase2 start: finalizeUnwrap
    1) should work with FOT tokens
...
     Error: VM Exception while processing transaction: reverted with
        custom error 'ERC20InsufficientBalance("0
        xc6e7DF5E7b4f2A278906862b61205850344D4e7d", 0, 9)'
...
    at SwapV0._refundUser (contracts/swap/swap_v0.sol:246)
```

Phase 2, the finalization, reverts with `ERC20InsufficientBalance` as expected. That happens when SwapV0 tries to process the refund, but since less tokens than `amountIn` are held by contract, the TX reverts.

Recommendation

When refunding the funds by re-wrapping them, check the actual balance of the SwapV0. If it is less than `amountIn`, then use the balance amount for wrapping.

Additionally, in order to support Uniswap swaps for FOT tokens, consider using swapExactTokensFor-TokensSupportingFeeOnTransferTokens

Client

Fix in c46a795a60342947db6f4b334d3fd42c6e3b33bb.

BurraSec

Fix verified. Actual balance of the SwapV0 is refunded if it is less than `amountIn`.

[H-2] Loss of funds when wrapping tokens with large total supply

Target

- WrapperUpgradeable.sol

Severity

- Impact: High
- Likelihood: Medium

Description

In `WrapperUpgradeable.wrap`, the contract transfers a specified amount of tokens from the user to itself, then mints an equivalent amount of cTokens (ERC7984) to the receiver. However, under certain conditions, the minted cToken amount can be zero due to a potential overflow in the cToken's total supply. This occurs within the `ERC7984Upgradeable._update` function:

```
  function _update(address from, address to, euint64 amount) internal virtual returns (euint64 transferred) {
      // ...

      if (from == address(0)) {
>>        (success, ptr) = FHESafeMath.tryIncrease($._totalSupply, amount);
          FHE.allowThis(ptr);
          _setTotalSupply(ptr);
      } else {
          // ...
      }

>>    transferred = FHE.select(success, amount, FHE.asEuint64(0));

      // ...
  }
```

As shown above, if the `tryIncrease` operation fails due to an overflow, the transferred amount defaults to zero, yet the mint operation still completes without reverting. This failure condition is triggered when adding amount would exceed the `uint64` maximum value.

Although seemingly unlikely, this risk must be considered seriously because `_totalSupply` uses `euint64`, the encrypted form of `uint64`. The maximum value for `uint64` is 18,446,744,073,709,551,615. Accounting for token decimals (maximum 6), the effective limit is approximately 18.44 trillion tokens.

Tokens exceeding this threshold include SHIB, PEPE, and BONK. If over 18.44 trillion units (excluding decimals) of any such token are wrapped, subsequent calls to wrap will mint zero cTokens, even though the underlying assets are successfully locked in `WrapperUpgradeable`.

This is especially relevant for SHIB, which has a circulating supply exceeding 589 trillion tokens and a market cap over 5 billion USD. The overflow condition would be triggered once only 3.2% of its supply is wrapped. Given SHIB's popularity, this is a plausible scenario.

While this currently affects mainly memecoins, the permissionless nature of cToken deployment increases the likelihood that other high-supply tokens will also encounter this issue. Moreover, new tokens with similarly large supplies may emerge in the future.

Once the overflow condition is met, all further wrap operations result in zero cTokens minted while still locking the user's assets, leading to direct and significant loss of funds.

Note: The same issue can also happen when `WrapperUpgradeable` tries to refund cTokens on `_finalizeSuccessfulUnwrap` and `_finalizeFailedUnwrap`.

Recommendation

Although the 64-bit total supply constraint is inherited from the OpenZeppelin implementation, `WrapperUpgradeable` should be modified to revert the transaction (or refund the user) when a zero cToken amount is minted due to a supply overflow. This ensures that user funds are not silently lost.

Client

Fixed in the following commits:

- 001e3dfddd282e322c5d8cccf6dd227ac9d4e1c9
- ec59b9d08e7e736a6f2a7e9058e12c3a68158efa
- 120d3f4a4dd370573a49e3ada8952cfaa44303f8

BurraSec

Fix verified.

[H-3] User's underlying asset may be stolen if the token's transfer has a hook

Target

- WrapperUpgradeable.sol

Severity

- Impact: High
- Likelihood: Medium

Description

In WrapperUpgradeable::wrap(), users can wrap their original ERC20 token to the related confidential ERC20 token.

In `_wrapERC20`, the actual transferred amount will be calculated according to the balance difference between current balance and balanceBefore.

The problem here is that if the original token has one callback in the transfer function(e.g ERC777), malicious users can re-enter the function `wrap` again because `wrap` function does not have one re-entrancy protection. Then malicious users can mint more cTokens via re-entrancy.

Although most ERC777 tokens are in-active, there are some active ERC777 tokens, e.g. VERA. There are 1700+ transactions in the latest 24 hours.

```
    function _wrapERC20(uint256 transferAmount_, uint256 totalFee_) private returns (uint64 mintAmount, uint256 actualFeeReceived) {
        uint256 balanceBefore = IERC20($._originalToken).balanceOf(address(this));
@>        IERC20($._originalToken).safeTransferFrom(msg.sender, address(this), transferAmount_);
        // calculate the balance difference between the transfer.
        uint256 balanceDifference = IERC20($._originalToken).balanceOf(address(this)) - balanceBefore;
        // recalculate the mint amount according to the
        mintAmount = SafeCast.toUint64(balanceDifference / rate);
    }
```

```
    function transferFrom(address holder, address recipient, uint256 amount) public override returns (bool) {
        require(recipient != address(0), "ERC777: transfer to the zero address");
```

```
        require(holder != address(0), "ERC777: transfer from the zero address");

        address spender = _msgSender();

        _callTokensToSend(spender, holder, recipient, amount, "", "");

        _move(spender, holder, recipient, amount, "", "");
        _approve(holder, spender, _allowances[holder][spender].sub(amount, "ERC777: transfer amount exceeds allowance"));

        _callTokensReceived(spender, holder, recipient, amount, "", "", false);

        return true;
    }
```

Recommendation

Add one re-entrancy protection in key functions.

Client

Fix in 4019928f465b96a731293aa9d0f72acb0e61d149.

BurraSec

Fix verified. Add `nonReentrant` modifier in function `wrap` to prevent re-entrancy.

[M-1] Sanctioned recipient addresses can cause funds to be unrecoverable

Target

- swap_v0.sol#L212

Severity

- Impact: High
- Likelihood: Low

Description

The confidential token system implements a two-phase unwrap process where users first burn confidential tokens and store swap parameters, then finalize the operation in a separate transaction after decryption. During this process, users specify a recipient address for the confidential tokens which gets immutably stored in the wrapper's storage. SwapV0 facilitates swaps between different confidential tokens by unwrapping to regular tokens, swapping via Uniswap, and wrapping the output back to confidential form.

`RegulatedERC7984Upgradeable` includes sanctions checking to comply with regulatory requirements. When minting or transferring tokens, the contract validates addresses against a sanctions list:

```
function _update(address from, address to, euint64 amount) internal
    override returns (euint64 transferred) {
    // ...
    if (to != address(0)) {
        _checkSanctions(sanctionsList, to);
        // ...
    }
}

function _checkSanctions(SanctionsList sanctionsList, address account)
    internal view {
    if (sanctionsList.isSanctioned(account)) {
        revert SanctionedAddress(account);
    }
}
```

When a user initiates a swap with a sanctioned recipient address, the swap executes successfully until SwapV0 attempts to wrap the output tokens. In `_handleUniswapSuccess()`:

```
function _handleUniswapSuccess(address[] memory path, uint256[] memory
    amounts, address to, uint256 unwrapRequestId) private {
    address tokenOut = path[path.length - 1];
    uint256 amountOut = amounts[amounts.length - 1];
    WrapperUpgradeable wrapperOut = coordinator.deployedWrappers(
        tokenOut);

    IERC20(tokenOut).forceApprove(address(wrapperOut), amountOut);
    wrapperOut.wrap(to, amountOut);  // 'to' is the sanctioned address
}
```

The `wrap()` function calls `confidentialToken.mint(to, amount)`, which triggers `_update()` and the sanctions check, causing a revert. This revert occurs outside any error handling in SwapV0.

The issue is that the recipient address is permanently stored during phase 1 of the unwrap. - User provides recipient in callback data when initiating unwrap - Address is stored in `WrapperUpgradeable._receivers[requestId].callbackData` - This data cannot be modified after storage - Every finalization attempt uses the same sanctioned address and reverts

The impact is high because user's confidential tokens are already burned, but there is no way for to recover the underlying tokens.

It is possible that recipient address was not sanctioned when user started the phase 1 of swap process, but it only got sanctioned just before finalization attempt. In that case, the sender did nothing wrong, but effectively lost all the funds he tried to swap.

Proof of Concept:

Add this test to `Swap.ts`:

```
it.only("should recover if receiver is sanctioned", async function () {
  const transferAmount = BigInt(1_000);
  const encryptedTransferAmount = await fhevm
    .createEncryptedInput(await cTokenA.getAddress(), signers.alice.address)
    .add64(transferAmount)
    .encrypt();

  const path = [await tokenA.getAddress(), await tokenB.getAddress()];

  const abiCoder = new ethers.AbiCoder();
  const callbackData = abiCoder.encode(
    ["address", "uint256", "address[]", "uint256", "address"],
    [await router.getAddress(), 0, path, Math.floor(Date.now() / 1000) + 6000, signers.bob.address],
  );
  const data = abiCoder.encode(["address", "bytes"], [await swapV0.getAddress(), callbackData]);

  // phase 1
  console.log("Phase1 start: confidentialTransferAndCall");
  const transferTx = await cTokenA
    .connect(signers.alice)
    [
      "confidentialTransferAndCall(address,bytes32,bytes,bytes)"
    ](await wrapperA.getAddress(), encryptedTransferAmount.handles[0], encryptedTransferAmount.inputProof, data);
  const transferReceipt = await transferTx.wait();
  console.log("Phase1 complete");

  // get sanctionsList from adminProvider
  const adminProviderAddress = await coordinator.adminProvider();
```

```
    const adminProvider = await ethers.getContractAt("AdminProvider",
        adminProviderAddress);
    const sanctionsListAddress = await adminProvider.sanctionsList();
    const sanctionsList = await ethers.getContractAt("SanctionsList",
        sanctionsListAddress);

    // Sanction Bob
    await sanctionsList.addToSanctionsList([signers.bob.address]);
    console.log("Bob (swap recipient sanctioned) after phase 1, but
        before phase 2");

    // phase 2
    console.log("Phase2 start: finalizeUnwrap");
    await getSwapFinalizedReceipt(transferReceipt, wrapperA, signers.
        bob);
    console.log("Phase2 complete");
  });
```

Running it:

```
npm run test

fhevm-hardhat-template@0.1.0 test
hardhat test

  Uniswap V2 Swap Test
Phase1 start: confidentialTransferAndCall
Phase1 complete
Bob (swap recipient sanctioned) after phase 1, but before phase 2
Phase2 start: finalizeUnwrap
    1) should recover if receiver is sanctioned
     Error: VM Exception while processing transaction: reverted with
         custom error 'SanctionedAddress("0
         x9965507D1a55bcC2695C58ba16FB37d819B0A4dc")'
```

We can see that finalization TX reverts, so underlying tokens cannot be recovered.

Recommendation

When the unwrap can't be finalized, the parity between cTokens and underlying tokens is broken. So instead of reverting when `to` is a sanctioned address, it might be better to gracefully handle it. SwapV0 can validate if `to` address is sanctioned before the swap is initiated. If recipient is sanctioned, trigger the refund mechanism.

However, note that in current implementation, the refund is issued to the same `to` address. So `wrap()` call in the `_refundUser` would revert as well. For that reason, but also in general, the better design would be to let the user provide the refund address along with other SwapV0 params. The main

difference is that users would provide `refund` addresses which they control, whereas `to` address is not necessarily controlled by the user. In the very unlikely case that even user-owned refund address gets sanctioned, you might consider moving the underlying tokens to the protocol fee recipient as a final fallback option.

Client

Fix in commits:

- 591d50e6fd884f0a90a02e31e3ad362023d6b637
- 26107fb4bb7634ca1902af7480c614e1916b28f7
- f4ac2dd3d84beae006f8895928c69a41811fb98f

BurraSec

Fix verified. If swap recipient is sanctioned, refund is triggered before swap takes place.

[M-2] Loss of fees due to rounding down in `FeeManager`

Target

- FeeManager.sol

Severity

- Impact: Medium
- Likelihood: Medium

Description

In `FeeManager`, the `getWrapFee` and `getUnwrapFee` functions round fees down due to integer division, which leads to precision loss and ultimately results in fee leakage.

```
    function getWrapFee(uint256 amount, address wrapFrom, address wrapTo) external view returns (uint256) {
        if (swapperFeeWaiverActive && hasRole(SWAPPER_ROLE, wrapFrom)) {
```

```
>>          return (amount * swapperWrapFeeBasisPoints) / MAX_BASIS_POINTS;
        }
>>      return (amount * wrapFeeBasisPoints) / MAX_BASIS_POINTS;
    }

    function getUnwrapFee(uint64 amount, address unwrapFrom, address unwrapTo) external view returns (uint64) {
        if (swapperFeeWaiverActive && hasRole(SWAPPER_ROLE, unwrapTo)) {
>>          return (amount * swapperUnwrapFeeBasisPoints) / MAX_BASIS_POINTS;
        }
>>      return (amount * unwrapFeeBasisPoints) / MAX_BASIS_POINTS;
    }
```

While the loss is typically negligible, it becomes exploitable in `getUnwrapFee`. This is because the `amount` parameter represents the decrypted value of burned confidential tokens, which generally have fewer decimals than the original tokens.

For example, a token like WBTC with 8 decimals will have its confidential counterpart (e.g., cWBTC) limited to 6 decimals. At a BTC price of 100k USD, one unit of cWBTC equates to 0.1 USD, enabling attackers to bypass fees through transaction splitting.

Example scenario:

- Attempting to unwrap 0.1 cWBTC, valued at 10k USD
- With a 0.1% fee, the expected unwrap fee is 10 USD
- By splitting the unwrapping into 101 smaller chunks, each rounding the fee to zero, the user avoids paying the full 10 USD.

Recommendation

Modify both `getWrapFee` and `getUnwrapFee` to round fees up instead of down.

Client

Fixed in a36421a557fb099d01e4608aa8d658cb15a828f1.

BurraSec

[M-3] Creation of ERC7984 and `WrapperUpgradeable` for non-existent tokens

Target

- DeploymentCoordinator

Severity

- Impact: Medium
- Likelihood: Medium

Description

The `DeploymentCoordinator` contract allows anyone to deploy a confidential token and wrapper by calling the `deploy` function, which is permissionless. Because `_tryGetAssetName`, `_tryGetAssetSymbol`, and `_tryGetAssetDecimals` are designed not to revert, this enables deployment for addresses that do not have any code yet.

This behavior can be exploited in a griefing attack scenario. If a project plans to deploy a token at a deterministic address using CREATE2, an attacker can front-run the deployment by calling `deploy` on that address before the actual token contract is live. As a result, the confidential token will be initialized with a default rate of **1e12**. If the actual token has a decimal count other than 18, this default rate will be incorrect, rendering both the confidential token and the wrapper unusable once the real token is deployed.

PoC

The following test demonstrates deployment of a confidential token and wrapper for a non-existent token, paste the following PoC in `Wrapper.ts`:

```
it("deploy non-existent token", async function () {
  // Test with a non-existent token address
  const nonExistentTokenAddress = "0x0000000000000000000000450000000000000007B";

  // Get deploy fee
```

```
  const adminProviderAddress = await coordinator.adminProvider();
  const adminProvider = await ethers.getContractAt("AdminProvider",
      adminProviderAddress);
  const feeManagerAddress = await adminProvider.feeManager();
  const feeManager = await ethers.getContractAt("FeeManager",
      feeManagerAddress);
  const deployTokenFee = await feeManager.getDeployFee(ethers.
      ZeroAddress);

  // Deploy directly with the address string
  const deployToken = await coordinator.connect(signers.charlie).deploy
      (nonExistentTokenAddress, { value: deployTokenFee });
  const receipt = await deployToken.wait();

  // Verify deployment succeeded
  const cTokenAddress = await coordinator.getConfidentialToken(
      nonExistentTokenAddress);
  const wrapperAddress = await coordinator.getWrapper(
      nonExistentTokenAddress);

  expect(cTokenAddress).to.not.equal(ethers.ZeroAddress);
  expect(wrapperAddress).to.not.equal(ethers.ZeroAddress);

  // Get the rate in cToken
  const cToken = await ethers.getContractAt("
      RegulatedERC7984Upgradeable", cTokenAddress);
  const rate = await cToken.rate();
  expect(rate).to.equal(BigInt(10) ** BigInt(12));
})
```

Recommendation

Update `deploy` to ensure the target token address contains some code before proceeding.

Alternatively, introduce access control so that only authorized entities can deploy new confidential tokens and wrappers.

Client

Fixed in 5cbc85702681a925b28a023a542602fd65b1b975.

BurraSec

Fix verified.

[M-4] Locked funds when unwraping if user uses Account Abstraction

Target

- WrapperUpgradeable.sol

Severity

- Impact: High
- Likelihood: Medium

Description

In `WrapperUpgradeable`, the `_finalizeSuccessfulUnwrap` function unconditionally executes a callback to the `to` address if the address contains code:

```
    function _finalizeSuccessfulUnwrap(
        uint256 requestId,
        uint64 actualBurnAmount,
        ReceiverEntry memory receiver
    ) private {
        // ...

>>      if (receiver.to.code.length > 0) {
>>          IWrapperReceiver(receiver.to).onUnwrapFinalizedReceived(msg.sender, unwrapAmount, requestId, receiver.callbackData);
        }
    }
```

This logic fails for users using Account Abstraction wallets. These wallets often do not implement the `onUnwrapFinalizedReceived` interface or support arbitrary function selectors. As a result, the callback will revert when `to` is an Account Abstraction wallet, preventing unwrapping from finalizing and causing funds to become locked within the `WrapperUpgradeable` contract.

Recommendation

Modify the logic to execute the callback only when the user explicitly provides non-empty callback data. This avoids unnecessary reverts for wallets that do not support the callback:

```
-       if (receiver.to.code.length > 0) {
+       if (receiver.to.code.length > 0 && receiver.callbackData.length > 0) {
```

```
            IWrapperReceiver(receiver.to).onUnwrapFinalizedReceived(msg.sender, unwrapAmount, requestId, receiver.callbackData);
        }
            IWrapperReceiver(receiver.to).onUnwrapFinalizedReceived(msg.sender, unwrapAmount, requestId, receiver.callbackData);
        }
```

Client

Fixed in 25659a5a92262c8dcb038a47ed64b320681fa55b.

BurraSec

Fix verified.

[M-5] Swap refunds triggered by low input in `getAmountsOut`

Target

- swap_v0.sol

Severity

- Impact: Medium
- Likelihood: Medium

Description

In `SwapV0.checkPath`, the function `getAmountsOut` is called on the Uniswap V2 router to validate the given path:

```
        // does the path exist with this router
>>      try IUniswapV2Router02(router).getAmountsOut(1, path) returns (uint256[] memory) {}
        catch Error(string memory reason) {
            return (false, reason, new bytes(0));
        } catch (bytes memory lowLevelData) {
            return (false, "", lowLevelData);
```

```
        }
```

However, using 1 wei as the `amountIn` argument can cause this call to revert even when the path is valid. This is due to how the router internally handles small amounts:

```
  function getAmountsOut(address factory, uint amountIn, address[] memory path) internal view returns (uint[] memory amounts) {
      require(path.length >= 2, 'UniswapV2Library: INVALID_PATH');
      amounts = new uint[](path.length);
      amounts[0] = amountIn;
      for (uint i; i < path.length - 1; i++) {
          (uint reserveIn, uint reserveOut) = getReserves(factory, path[i], path[i + 1]);
          amounts[i + 1] = getAmountOut(amounts[i], reserveIn, reserveOut);
      }
  }

  function getAmountOut(uint amountIn, uint reserveIn, uint reserveOut) internal pure returns (uint amountOut) {
>>    require(amountIn > 0, 'UniswapV2Library: INSUFFICIENT_INPUT_AMOUNT');
      require(reserveIn > 0 && reserveOut > 0, 'UniswapV2Library: INSUFFICIENT_LIQUIDITY');
      uint amountInWithFee = amountIn.mul(997);
      uint numerator = amountInWithFee.mul(reserveOut);
      uint denominator = reserveIn.mul(1000).add(amountInWithFee);
      amountOut = numerator / denominator;
  }
```

Since the router loops over the path and repeatedly applies `getAmountOut`, an intermediate `amountOut` can round down to zero due to the initial `amountIn` of 1 wei, causing a revert with "INSUFFICIENT_INPUT_AMOUNT". This leads `checkPath` to incorrectly report a valid path as invalid, resulting in unnecessary refunds. Users attempting a swap lose any wrap or unwrap fees incurred, with no successful transaction.

Recommendation

Remove the `getAmountsOut` call from `checkPath`. Let the swap attempt determine validity; if it fails, the refund mechanism will still trigger appropriately.

```
    function checkPath(WrapperUpgradeable wrapper, address router, address[] memory path) public view returns (bool, string memory, bytes memory) {
        // ...

-       // does the path exist with this router
```

```
-       try IUniswapV2Router02(router).getAmountsOut(1, path) returns (uint256[] memory) {}
-       catch Error(string memory reason) {
-           return (false, reason, new bytes(0));
-       } catch (bytes memory lowLevelData) {
-           return (false, "", lowLevelData);
-       }

        return (true, "", new bytes(0));
    }
```

Client

Fixed in 59f9481ccf5cc4d3cee1f9b65dd17a31d5d65f1e.

BurraSec

Fix verified.

[M-6] Arbitrary `routerAddress` allows fund theft and fee bypass

Target

- swap_v0.sol

Severity

- Impact: Medium
- Likelihood: Medium

Description

The `SwapV0` contract accepts a user-provided `routerAddress` for executing swaps, without validating whether the address corresponds to a legitimate Uniswap V2 router. This opens the door to several exploits:

1. **Fund theft via manipulated `amounts` array**

A user can supply a custom `routerAddress` that returns a maliciously crafted amounts array, tricking the contract into transferring out all of its token balances. While `SwapV0` is not intended to hold funds between calls, any tokens temporarily held during execution would be at risk.

2. **Fund theft via retained allowance**

The following line grants an allowance to the user-supplied router:

```
    IERC20(path[0]).forceApprove(routerAddress, amountIn);
```

An attacker could deploy a router that neither performs a transfer nor a swap. By using a path where `path[0] == path[length - 1]`, the tokens are returned post-swap and the allowance remains. If tokens are later deposited into `SwapV0`, the attacker can exploit the residual approval to steal funds.

3. **Bypass wrap fees**

An attacker could exploit the reduced fees in `SwapV0` by unwrapping a minimal amount (e.g., 1 wei of an ERC7984 token). The custom router could simulate a swap and transfer a large token amount (e.g., 10 ETH) to `SwapV0`, which would then wrap the full amount with reduced fees.

4. **Bypass unwrap fees**

Similarly, an attacker could unwrap a large amount of cTokens (e.g., 10 cETH) via a custom router that redirects tokens directly to the user and returns zero tokens to the contract. This causes `_handleUniswapSuccess` to execute with no real output handling, bypassing intended fee logic.

Recommendation

Validate that the `routerAddress` provided by the user corresponds to a legitimate, trusted Uniswap V2 router. Avoid allowing arbitrary router execution without verification.

Client

Fixed in ec129155170adb3ee1d8ba5218bf1dd67ba861b6.

BurraSec

Fix verified.

[M-7] Sanctioned entities can still operate token actions as approved operators

Target

- RegulatedERC7984Upgradeable.sol

Severity

- Impact: Medium
- Likelihood: Medium

Description

The documentation relative to sanctions enforcement states that sanctioned addresses must be completely restricted from participating in any token operations:

```
Sanctions Enforcement
    **Check Points**: All mint, burn, transfer, and approve operations
    **Implementation**: `_checkSanctions(address)` queries AdminProvider.sanctionsList
    **Effect**: Sanctioned addresses cannot participate in any token operations
    **Bypass**: No bypass mechanisms - enforcement is absolute
```

However, in the `RegulatedERC7984Upgradeable` implementation, only the `from` and `to` addresses are validated against the sanctions list. The `msg.sender` (operator) is not checked, which allows a sanctioned address to operate on behalf of non-sanctioned users and initiate token transfers.

```
  function _update(address from, address to, euint64 amount) internal override returns (euint64 transferred) {
      // ...

      // Grant admin access to transferred amount and updated balances
      if (from != address(0)) {
>>        _checkSanctions(sanctionsList, from);
          euint64 fromBalance = confidentialBalanceOf(from);
          FHE.allow(fromBalance, cachedRegulator);
          if (cachedTokenRegulator != address(0)) {
              FHE.allow(fromBalance, cachedTokenRegulator);
          }
      }
      if (to != address(0)) {
```

```
>>          _checkSanctions(sanctionsList, to);
            euint64 toBalance = confidentialBalanceOf(to);
            FHE.allow(toBalance, cachedRegulator);
            if (cachedTokenRegulator != address(0)) {
                FHE.allow(toBalance, cachedTokenRegulator);
            }
        }

        // ...
    }
```

This gap directly contradicts the stated policy of absolute enforcement and introduces a compliance risk, as sanctioned entities can continue engaging with the token indirectly.

Recommendation

Update `_update` in `RegulatedERC7984Upgradeable` to include a sanctions check on `msg.sender`, preventing sanctioned operators from initiating token operations on behalf of others.

Client

Fixed in 16643f7e9d6e226c139ee379ee3829782ee5012f.

BurraSec

Fix verified.

[M-8] Permissionless finalizeUnwrap can allow anyone to trigger a forced refund

Target

- WrapperUpgradeable.sol

Severity

- Impact: Medium
- Likelihood: Medium

Description

The function finalizeUnwrap within Wrapper and WrapperUpgradeable is permissionless, which allows anyone to call it given that the correct proof is provided.

However, if the unwrap is designed to execute a swap on the swapper and the slippage params are tight, the user might want to wait a bit before calling finalizeUnwrap so that the swap is correctly executed. Also, if the original token is paused for whatever reason, the unwrap would also fail.

But given that finalizeUnwrap is permissionless, anyone can call finalizeUnwrap at a bad time, causing the whole opeartion to fail and triggering a refund to the initial user. This causes the user a loss of gas (as the unwrap has to be retried) as well as a loss in wrap/unwrap fees. In case of a failing swap, the loss of fees only happens if swapperFeeWaiverActive is false, or swapperWrapFeeBasisPoints and swapperUnwrapFeeBasisPoints are non-zero.

Recommendation

Only allow the initiator of an unwrap to call finalizeUnwrap.

Alternatively, allow the initiator to provide a trusted address, which can call finalizeUnwrap as well.

Client

Fixed in d645e4902ef478c9a0756fd2c534d80ae2a4b956.

BurraSec

Fixed that only granted operator can trigger `finalizeUnwrap`.

[M-9] Custom token route enables wrap/unwrap fee circumvention

Target

- swap_v0.sol

Severity

- Impact: Medium
- Likelihood: Medium

Description

The `SwapV0` contract supports wrapping and unwrapping confidential tokens, which incur wrap and unwrap fees. However, these fees can be reduced or waived when the operation follows a specific sequence: unwrap → swap → wrap. This optimization is intended for legitimate token swaps using the swapper contract as an intermediary, as defined in `FeeManager`:

```
function getWrapFee(uint256 amount, address wrapFrom, address wrapTo) external view returns (uint256) {
    if (swapperFeeWaiverActive && hasRole(SWAPPER_ROLE, wrapFrom)) {
>>      return (amount * swapperWrapFeeBasisPoints) / MAX_BASIS_POINTS;
    }
    return (amount * wrapFeeBasisPoints) / MAX_BASIS_POINTS;
}


function getUnwrapFee(uint64 amount, address unwrapFrom, address unwrapTo) external view returns (uint64) {
    if (swapperFeeWaiverActive && hasRole(SWAPPER_ROLE, unwrapTo)) {
>>      return (amount * swapperUnwrapFeeBasisPoints) / MAX_BASIS_POINTS;
    }
    return (amount * unwrapFeeBasisPoints) / MAX_BASIS_POINTS;
}
```

The intention is to incentivize end-to-end swaps between confidential tokens. However, an attacker can game this logic to conduct simple wrap or unwrap operations while still benefiting from the reduced fee structure.

Example 1: Avoid wrap fees:

1. Bob wants to wrap 10 WETH into cWETH while paying only the reduced swapper fee.
2. He deploys a custom ERC20 token and creates two pools on Uniswap v2: `WETH<>fakeToken` and `USDC<>fakeToken`.
3. He obtains 1 wei of cUSDC and unwraps it, setting the receiver to the swapper. Since the amount is negligible, the unwrap fee is effectively zero.
4. The swapper receives 1 wei of USDC, and the swap path is decoded as `[USDC, fakeToken, WETH]`.
5. 1 wei of USDC is swapped to `fakeToken`, then to WETH. The swapper receives 10 WETH as the swap output.
    - This step is possible because Bob controls all liquidity in these pools, so he can set the desired spot price.
6. The swapper wraps 10 WETH into cWETH, applying only the reduced wrap fee due to its role.

7. In the end, Bob has avoided the unwrap fee (due to the negligible amount), avoids swap fees (as he controls all liquidity in the pools), and receives cWETH with a reduced wrap fee.

Example 2: Avoid unwrap fees:

1. Bob wants to unwrap 10 cWETH into WETH with the reduced swapper fee.
2. He deploys a fake token and creates two liquidity pools: `WETH<>fakeToken` and `USDC<>fakeToken`.
3. Bob unwraps 10 cWETH, setting the receiver to the swapper, thus incurring only the reduced unwrap fee.
4. The swapper receives 10 WETH and executes a swap along the path `[WETH, fakeToken, USDC]`.
5. The WETH is swapped to fakeToken, then to USDC, returning only 1 wei of USDC to the swapper.
    - This step is possible because Bob controls all liquidity in these pools, so he can set the desired spot price.
6. The swapper wraps 1 wei of USDC into cUSDC, effectively incurring no wrap fee.
7. Bob later withdraws liquidity from the fake pools, recovering his 10 WETH.
8. In the end, Bob has only paid for the reduced unwrap fee, avoids swap fees (as he controls all liquidity in the pools), and avoids the wrap fee (due to the negligible amount).

Recommendation

Two mitigation options: 1. Remove the fee reduction for the swapper role and apply standard fees to all users. 2. Introduce a whitelist for valid tokens in the swap path and enforce checks to ensure all tokens in the path are whitelisted.

Client

Fixed in ba3d9074e5b950cd18542af991962cd0208af379.

BurraSec

Fix verified.

[M-10] Missing one interface to update FHE's coprocessor

Target

- WrapperUpgradeable.sol

Severity

- Impact: Medium
- Likelihood: Medium

Description

In WrapperUpgradeable::initialize, the FHE will be configured with the default configuration from `getEthereumCoprocessorConfig`.

The problem here is that currently zamaConfig library will provide valid config for `Sepolia` chain. All addresses for Ethereum Chain are address(0).

If the WrapperUpgradeable is deployed based on current implementation, FHE will not work as expected because all core configurations are address(0). Even if zama team deploys the related addresses, the contract `WrapperUpgradeable` misses one interface to update the addresses to the correct addresses.

```
    function initialize(
        address originalToken_,
        RegulatedERC7984Upgradeable confidentialToken_,
        AdminProvider adminProvider_,
        address admin_
    ) public initializer {
        __EthereumConfig_init();
    }
    function __EthereumConfig_init_unchained() internal onlyInitializing {
        FHE.setCoprocessor(ZamaConfig.getEthereumCoprocessorConfig());
    }
    function _getEthereumConfig() private pure returns (CoprocessorConfig memory) {
        // The addresses below are placeholders and should be replaced with actual addresses
        // once deployed on the Ethereum mainnet.
        return
            CoprocessorConfig({ACLAddress: address(0), CoprocessorAddress: address(0), KMSVerifierAddress: address(0)});
```

```
    }
```

Recommendation

Add one interface to update the correct coprocessor.

Client

Acknowledge this issue.

BurraSec

The Zaffier team will use and deploy the related contract only after the Zama team's team and the related contracts are deployed.

[M-11] Users may lose funds if transfer fails

Target

- WrapperUpgradable.sol

Severity

- Impact: High
- Likelihood: Low

Description

In `_finalizeSuccessfulUnwrap`, if the underlying token transfer fails, e.g. the ERC20 contract is paused, the original confidential tokens will be reminted to `to`. This design is accepted on condition that the `to` is the actual receiver.

The problem here is that if users want to swap confidential tokens via this unwrap. The `to` address will be `swap_v0` contract. Then original confidential tokens will be minted to `swap_v0`. However, `swap_v0` is not the actual expected `recipient`. Users will lose all funds.

```
    function _finalizeSuccessfulUnwrap(
        uint256 requestId,
        uint64 actualBurnAmount,
        ReceiverEntry memory receiver
    ) private {
        bool unwrapSuccess = _transferUnderlying($._originalToken,
            receiver.to, unwrapAmount);
        if (unwrapSuccess == false) {
            unwrapAmount = 0;
            if (feeSuccess == false) {
                // Mint everything back to user if both transfers
                    failed
                $._confidentialToken.mint(receiver.to, actualBurnAmount
                    );
                feeAmount256 = 0;
            } else {
                $._confidentialToken.mint(receiver.to, actualBurnAmount
                    - feeAmount64);
            }
        }
    }
```

SwapV0 itself has very similar issue. The refund in _refundUser is issued to the to destination address. But that's not ideal for multiple reasons :

- `to` might not be the end receiver, but intermediate contract as it's the case in this report
- refund mechanism is fallback path, so doesn't make really sense to enforce the destination for fallback recovery is the same as original destination
- since refund is a fallback path, user should have option to provide address he controls which not necessary the case for the original destination address

Recommendation

Let users specify a refund address in WrapperUpgradable and swap_v0 failure path.

Client

Fixed in the following commits:

- 441ef03bb7064eb84bc21140cb2bdeae17b3febd
- b8b9d9811b3fb0e3cfd884b1dc387f521d9cae7e
- 93286788ccd878af55af86c7ea2cb11fa4acbc69
- 2e3eabfba6889002500e8668ad2088a2b6a2c029

BurraSec

Fix verified. When transfer fails, the confidential token will be re-minted to `refund` address defined by the user.

[L-1] Removal of leading zeroes when deriving name/symbol from address

Target

- DeploymentCoordinator.sol

Severity

- Impact: Low
- Likelihood: Low

Description

The helper functions `_tryGetAssetName` and `_tryGetAssetSymbol` fall back to using the token address when the token does not implement `name` or `symbol`, or when those calls fail. This fallback converts the address to a hex string using `Strings.toHexString` after casting the address to `uint160`:

```
    function _tryGetAssetName(address token_) private view returns (string memory) {
        // ...
        // Fallback: full address as string (40 hex chars)
>>      return Strings.toHexString(uint160(token_));
    }

    function _tryGetAssetSymbol(address token_) private view returns (string memory) {
        // ...
        // Fallback: first 6 chars of address hex string
>>      string memory fullAddress = Strings.toHexString(uint160(token_));
        return _substring(fullAddress, 0, 8); // "0x" + 6 chars
    }
```

However, `Strings.toHexString(uint256)` from OpenZeppelin returns a minimal-length hex string, omitting leading zero bytes. This leads to two issues:

- For tokens at low numeric addresses, the fallback name or symbol is not the standard 40-character hex representation.
- For addresses with a numeric value less than 2^{16} (i.e., with at least 36 leading zero hex digits), the resulting string can be shorter than 8 characters. In such cases, the call to `_substring` reverts due to the bounds check:

```
    function _substring(string memory str, uint256 startIndex, uint256 endIndex) private pure returns (string memory) {
        bytes memory strBytes = bytes(str);
>>      require(endIndex <= strBytes.length && startIndex < endIndex, "Invalid substring indices");

        // ...
    }
```

As a result, `_tryGetAssetSymbol` can revert for tokens deployed at very low addresses or for intentionally crafted vanity addresses.

Recommendation

Use a fixed-width hex representation for addresses in the fallback path to ensure the string is always at least 8 characters long and preserves leading zeros:

```
-       return Strings.toHexString(uint160(token_));
+       return Strings.toHexString(token_);
```

Client

Fixed in 68b3cdd0bf971a4252dd9d90f6b6dcf30f9a9012.

BurraSec

Fix verified.

[L-2] Contracts fail to reflect updated `adminProvider` from `DeploymentCoordinator`

Target

- RegulatedERC7984Upgradeable.sol
- WrapperUpgradeable.sol

Severity

- Impact: Low
- Likelihood: Low

Description

In `DeploymentCoordinator`, the `setAdminProvider` function allows the `AdminProvider` contract address to be updated. Since `AdminProvider` is not upgradeable, it must be re-deployed and its reference updated via `DeploymentCoordinator`.

```
    function setAdminProvider(AdminProvider adminProvider_) external onlyOwner {
        require(address(adminProvider_) != address(0), ZeroAddressAdminProvider());
        address oldAdminProvider = address(adminProvider);
        adminProvider = adminProvider_;
        emit AdminProviderUpdated(oldAdminProvider, address(adminProvider_));
    }
```

However, when this update occurs, existing deployments of `WrapperUpgradeable` and `RegulatedERC7984Upgradeable` retain outdated references to the previous `adminProvider`. These contracts currently lack the functionality to update their `adminProvider` references, requiring each to be upgraded individually. This introduces unnecessary operational overhead and additional gas costs.

Recommendation

Refactor `WrapperUpgradeable` and `RegulatedERC7984Upgradeable` to retrieve the `AdminProvider` address dynamically from `DeploymentCoordinator`, instead of storing it as a fixed reference.

Client

Fixed in ef9b98ce248835f9712de130ebdf6c46b6e85fdf.

BurraSec

Fix verified.

[L-3] Invalid router address causes permanent fund loss

Target

- swap_v0.sol#L108

Severity

- Impact: Med
- Likelihood: Low

Description

The confidential token swap system uses a two-phase process where users burn confidential tokens and commit swap parameters in phase 1, then execute the swap after decryption in phase 2. Users must specify a Uniswap V2 router address as part of their swap configuration, which gets permanently stored in the wrapper's storage. The system fails if users provide an address that contains no contract code.

During the swap flow, users encode the router address in their callback data:

```
bytes memory swapData = abi.encode(
    routerAddress,  // User-provided router
    amountOutMin,
    path,
    deadline,
    recipient
);
```

This data is immutably stored during phase 1 in `WrapperUpgradeable.onConfidentialTransferReceived()`:

```
$._receivers[requestId] = ReceiverEntry({
    to: SwapV0,
    callbackData: swapData,  // Contains router address, cannot be changed
    expectedBurnAmount: amount,
    actualBurnAmount: actualBurnAmount
});
```

When finalization occurs, SwapV0 attempts to validate the swap path in checkPath():

```
function checkPath(WrapperUpgradeable wrapper, address router, address[] memory path) public view {
    // ...
    try IUniswapV2Router02(router).getAmountsOut(1, path) returns (uint256[] memory) {
        // Valid router
    } catch Error(string memory reason) {
        return (false, reason, new bytes(0));
    } catch (bytes memory lowLevelData) {
        return (false, "", lowLevelData);
    }
}
```

The issue is that when router is an EOA with no code, Solidity's external interface call `IUniswapV2Router02(router).getAmountsOut()` causes an immediate revert that cannot be caught by try-catch. As a consequence, `finalizeUnwrap` TX reverts. User's confidential tokens are already burned from phase 1, while underlying tokens are stuck in wrapper and cannot be recovered.

Proof of Concept:

Add this test case to `Swap.ts`:

```
  it.only("should recover if router address is incorrect", async function () {
    const transferAmount = BigInt(1_000);
    const encryptedTransferAmount = await fhevm
      .createEncryptedInput(await cTokenA.getAddress(), signers.alice.address)
      .add64(transferAmount)
      .encrypt();

    const path = [await tokenA.getAddress(), await tokenB.getAddress()];

    // @audit set incorrect router address
    const incorrectRouterAddress = "0x0000000000000000000000000000000000001388";

    const abiCoder = new ethers.AbiCoder();
    const callbackData = abiCoder.encode(
```

```
      ["address", "uint256", "address[]", "uint256", "address"],
      // @audit incorrect router address passed in
      [incorrectRouterAddress, 0, path, Math.floor(Date.now() / 1000) +
          6000, signers.alice.address],
    );
    const data = abiCoder.encode(["address", "bytes"], [await swapV0.
        getAddress(), callbackData]);

    // phase 1
    console.log("Phase1 start: confidentialTransferAndCall");
    const transferTx = await cTokenA
      .connect(signers.alice)
      [
        "confidentialTransferAndCall(address,bytes32,bytes,bytes)"
      ](await wrapperA.getAddress(), encryptedTransferAmount.handles
          [0], encryptedTransferAmount.inputProof, data);
    const transferReceipt = await transferTx.wait();
    console.log("Phase1 complete");

    // phase 2
    console.log("Phase2 start: finalizeUnwrap");
    await getSwapFinalizedReceipt(transferReceipt, wrapperA, signers.
        bob);
    console.log("Phase2 complete");
  });
```

Run it:

```
npm run test

> fhevm-hardhat-template@0.1.0 test
> hardhat test

  Uniswap V2 Swap Test
Phase1 start: confidentialTransferAndCall
Phase1 complete
Phase2 start: finalizeUnwrap
    1) should recover if router address is incorrect


  0 passing (198ms)
  1 failing

  1) Uniswap V2 Swap Test
       should recover if router address is incorrect:
     Error: Transaction reverted: function returned an unexpected
         amount of data
```

As we can see, finalization reverted, instead of issue being gracefully handled and tokens refunded.

Recommendation

Add an explicit code size check before attempting any interface calls on the router address. If there is no code, trigger the refund path. For example:

```
function checkPath(WrapperUpgradeable wrapper, address router, address
    [] memory path) public view {
    // Add this check first
    if (router.code.length == 0) {
        return (false, "Router has no code", "");
    }

    // Now safe to call interface methods
    try IUniswapV2Router02(router).getAmountsOut(1, path) returns (
        uint256[] memory) {
        // ...
    } catch {
        // ...
    }
}
```

Client

Fix in ec129155170adb3ee1d8ba5218bf1dd67ba861b6.

BurraSec

Fix verified. Router whitelisting is implemented.

[L-4] Zero address swap recipient causes token-cToken parity break

Target

- swap_v0.sol#L212

Severity

- Impact: Medium
- Likelihood: Low

Description

The confidential token swap system stores user-provided recipient addresses immutably during the unwrap initiation phase. When users accidentally or intentionally set the recipient to address(0), the entire swap operation becomes permanently unrecoverable due to a revert in the final minting step.

During swap initiation, users encode their desired recipient in the callback data:

```
bytes memory swapData = abi.encode(
    routerAddress,
    amountOutMin,
    path,
    deadline,
    address(0)  // Zero address as recipient
);
```

This data gets permanently stored in phase 1 and cannot be modified. The swap executes successfully in phase 2, with SwapV0 receiving the output tokens. However, when attempting to wrap these tokens in `_handleUniswapSuccess()`:

```
function _handleUniswapSuccess(...) {
    // Swap succeeded, SwapV0 has output tokens
    WrapperUpgradeable wrapperOut = coordinator.deployedWrappers(
        tokenOut);

    IERC20(tokenOut).forceApprove(address(wrapperOut), amountOut);
    wrapperOut.wrap(address(0), amountOut);  // Attempts to mint to
        zero address
}
```

The `wrap()` call triggers `RegulatedERC7984Upgradeable._mint()`, which explicitly prevents minting to the zero address:

```
function _mint(address to, euint64 amount) internal returns (euint64) {
    require(to != address(0), ERC7984InvalidReceiver(address(0)));  //
        REVERTS
    return _update(address(0), to, amount);
}
```

This revert occurs outside any try-catch blocks in SwapV0, causing the entire finalization to fail. Since the recipient address is immutably stored in the wrapper's callback data, every finalization attempt will hit the same revert. The impact is underlying funds left stuck - confidential tokens are burned, but output tokens can never be distributed, remaining locked in the wrapper contract indefinitely.

Recommendation

SwapV0 should validate the recipient address immediately upon receiving the callback. Add validation and trigger the refund mechanism if provided `to` address is address(0), ideally to the user-provided refund address.

Client

Fix in commit 4eb7a224b4a067ae8767ba8e48105516e9a73b4b.

BurraSec

Fix verified. Refund is issued if swap recipient is address(0).

[L-5] Transfer batcher doesn't allow operators

Target

- ERC7984TransferBatcher.sol

Severity

- Impact: Low
- Likelihood: Medium

Description

The `ERC7984TransferBatcher` contract enables users to perform batch transfers of a single ERC7984 token to reduce gas costs. However, it restricts this functionality solely to token owners and does not support operators acting on behalf of others:

```
function confidentialBatchTransfer(
    RegulatedERC7984Upgradeable cToken,
    ConfidentialTransferInput[] calldata transfers
) external payable {
    // ...

    for (uint256 i = 0; i < transfers.length; i++) {
        // ...
```

```
            cToken.confidentialTransferFrom(
>>              msg.sender,
                transfers[i].to,
                amount
            );

            // ...
        }
    }
```

As shown, `msg.sender` is always passed as the sender in each call to `confidentialTransferFrom`. This prevents approved operators from initiating batch transfers on behalf of token owners.

For example, if Bob approves Alice as an operator, Alice can use `confidentialTransferFrom` directly to transfer Bob's tokens. However, she is not permitted to use `confidentialBatchTransfer` for the same purpose, despite being properly authorized.

This limits flexibility and utility for users relying on delegated permissions.

Recommendation

Consider whether approved operators should be allowed to perform batch transfers on behalf of token holders. If so, modify `confidentialBatchTransfer` to support this capability in line with the standard operator model.

Client

Fixed in 624872343f6aaee03e709d8709f164e62f905e00

BurraSec

Fix verified.

[L-6] Potential stealing funds from `SwapV0`

Target

- swap_v0.sol

Severity

- Impact: Low
- Likelihood: Medium

Description

It is possible for an attacker to steal tokens held in the `SwapV0` contract through a crafted contract interaction:

1. The attacker calls `SwapV0.onUnwrapFinalizedReceived` using a contract they control that implements the `originalToken` and `wrap` functions.
2. During `checkPath`, `originalToken` is invoked on `msg.sender`. The attacker's contract returns a token address different from `path[0]`, triggering a refund:

```
address underlyingAddress = wrapper.originalToken();

// does the cToken underlying correspond to the input path token
if (underlyingAddress != path[0]) {
    return (false, "", INPUT_PATH_IS_NOT_UNDERLYING);
}
```

3. In `_refundUser`, `originalToken` is called again, returning a valid token address:

```
WrapperUpgradeable wrapperIn = WrapperUpgradeable(payable(msg.sender));
address tokenIn = wrapperIn.originalToken();
```

4. The contract then approves `tokenIn` to `wrapperIn` (controlled by the attacker) and calls `wrap`. The attacker can make `wrap` succeed without taking any action.

As a result, `SwapV0` ends up approving tokens to an attacker-controlled contract, which can later extract those tokens.

While this does not pose an immediate threat due to the expected lack of funds in `SwapV0`, it still represents a valid attack vector if any tokens are ever sent there and not taken out before the txn ends.

Recommendation

Update the `onUnwrapFinalizedReceived` function to verify that the caller is a legitimate wrapper. If not, revert the transaction.

Client

Fixed in following commits:

- 03f3320726ccab2a385e59f52a570dccdfc3b71d
- 51e028fcad798b32c4cc169ed53497ac4b971d51
- e8f84a4db54a41d672ad22de88fa4d5faa3b8f30.

BurraSec

Fix verified.

[L-7] Callback return value ignored

Target

- WrapperUpgradeable.sol#L359

Severity

- Impact: Low
- Likelihood: Medium

Description

When `finalizeUnwrap` calls the receiver's `onUnwrapFinalizedReceived` callback, the return value is completely ignored:

```
if (receiver.to.code.length > 0) {
    IWrapperReceiver(receiver.to).onUnwrapFinalizedReceived(
        msg.sender, unwrapAmount, requestId, receiver.callbackData
    );
}
```

`IWrapperReceiver` documents return value as:

```
* @return A boolean indicating success of the callback operation
```

And `Swap0` as:

```
/// @dev Returns encrypted bool to allow wrapper to track success/
    failure on-chain
```

For example, SwapV0 will return true if swap was successful and false if refund was triggered. But wrapper does not use the return value which hurts the integration possibilities. If Wrapper processed the callback's return value and returned it itself from `finalizeUnwrap`, the smart contract caller could act on the result. Ie. swap failed due to the slippage - caller can revert the TX and try again in a minute instead of getting the refund and having to initiate unwrap from scratch.

Recommendation

Return the callback result from finalizeUnwrap:

```
function finalizeUnwrap(
    uint256 requestId,
    bytes memory abiEncodedClearBurnAmounts,
    bytes memory decryptionProof
) external virtual returns (bool) {
    // ... existing logic ...

    bool callbackSuccess = true;
    if (receiver.to.code.length > 0) {
        callbackSuccess = IWrapperReceiver(receiver.to).
            onUnwrapFinalizedReceived(
             msg.sender, unwrapAmount, requestId, receiver.callbackData
        );
    }
    return callbackSuccess;
}
```

Client

Fix in commits:

- 535ab5160239d712e12d93464733d84e92265376
- 69de0b937d4f2116e93959a1c0067e417ad371f1

BurraSec

Fix verified. Return value is passed on to the caller.

[I-1] Inaccurate `actualFeeReceived` due to unaccounted `transferDust`

Target

- WrapperUpgradeable.sol

Severity

- Impact: Low
- Likelihood: Low

Description

In `WrapperUpgradeable._wrapERC20`, the `actualFeeReceived` is calculated by measuring the fee recipient's balance before and after the `safeTransferFrom` call. This approach correctly accounts for tokens with transfer fees. However, it overlooks the `transferDust` value, which is also transferred to the fee recipient just prior to the fee transfer.

```
    function _wrapERC20(uint256 transferAmount_, uint256 totalFee_) private returns (uint64 mintAmount, uint256 actualFeeReceived) {
        // ...

        // Transfer dust
        uint256 transferDust = balanceDifference % rate;
        if (transferDust > 0) {
>>          IERC20($._originalToken).safeTransfer(feeRecipient, transferDust);
        }

        // Transfer fee and track actual received
        uint256 feeBalBefore = IERC20($._originalToken).balanceOf(feeRecipient);
>>      IERC20($._originalToken).safeTransferFrom(msg.sender, feeRecipient, totalFee_);
        actualFeeReceived = IERC20($._originalToken).balanceOf(feeRecipient) - feeBalBefore;
    }
```

Since the balance snapshot is taken after the dust transfer, the `actualFeeReceived` value underreports the total amount received by the fee recipient. This results in a slightly inaccurate value being emitted in the Wrapped event:

```
    emit Wrapped(mintAmount, amount_, actualFeeReceived, to_, mintTxId);
```

Recommendation

Move the `feeBalBefore` snapshot to occur before the dust transfer:

```
    function _wrapERC20(uint256 transferAmount_, uint256 totalFee_) private returns (uint64 mintAmount, uint256 actualFeeReceived) {
        // ...

+       uint256 feeBalBefore = IERC20($._originalToken).balanceOf(feeRecipient);

        // Transfer dust
        uint256 transferDust = balanceDifference % rate;
        if (transferDust > 0) {
            IERC20($._originalToken).safeTransfer(feeRecipient, transferDust);
        }

        // Transfer fee and track actual received
-       uint256 feeBalBefore = IERC20($._originalToken).balanceOf(feeRecipient);
        IERC20($._originalToken).safeTransferFrom(msg.sender, feeRecipient, totalFee_);
        actualFeeReceived = IERC20($._originalToken).balanceOf(feeRecipient) - feeBalBefore;
    }
```

Client

Fixed in 34ea09a8e4a67df94c0fe4b59bba87a1cdfb075e.

BurraSec

Fix verified.

[I-2] cETH is not supported in swap_v0

- swap_v0.sol

Severity

- Impact: Low
- Likelihood: Low

Description

`swap_v0` contract aims to enables atomic swaps between confidential tokens via Uniswap V2.

The work flow is as below:

User initiates confidentialTransferAndCall → unwrap → swap → wrap

The problem here is that the `cETH` cannot be swapped via `swap_v0`. When users finalize the unwrap, the `cETH`'s underlying token, native Ether will be transferred to `swap_v0`. The `swap_v0` does not own one `receive`/`fallback` interface to receive native Ether, and in function `onUnwrapFinalizedReceived`, the underlying token is assumed as one ERC20 token.

```
    function _finalizeSuccessfulUnwrap(
        uint256 requestId,
        uint64 actualBurnAmount,
        ReceiverEntry memory receiver
    ) private {
        bool unwrapSuccess = _transferUnderlying($._originalToken, receiver.to, unwrapAmount);
    }
    function onUnwrapFinalizedReceived(
        address /* operator */,
        uint256 amountIn,
        uint256 unwrapRequestId,
        bytes calldata data
    ) external returns (bool) {
        IERC20(path[0]).forceApprove(routerAddress, amountIn);

        try IUniswapV2Router02(routerAddress).swapExactTokensForTokens(
            amountIn,
            amountOutMin,
            path,
            address(this),
            deadline
        ) returns (uint256[] memory amounts) {
            _handleUniswapSuccess(path, amounts, to, unwrapRequestId);
        } catch Error(string memory reason) {
            returnVal = _refundUser(routerAddress, path, amountIn, to, unwrapRequestId, reason, new bytes(0));
        } catch (bytes memory lowLevelData) {
            returnVal = _refundUser(routerAddress, path, amountIn, to, unwrapRequestId, "", lowLevelData);
```

```
            }
        }
```

Recommendation

Clearly document cETH is not supported in code natspec and official docs if cETH is not planned to be supported.

Client

Fixed in 99e6297d0d18666cfc6db3d177fb0fbb38009cbe.

BurraSec

Fix verified. Support cETH swap via adding `swapExactTokensForETH` and `swapExactETHForTokens`.

[I-3] Batcher retry mechanism leaks info about failed transactions

Target

- ERC7984TransferBatcher.sol#L109

Severity

- Impact: Low
- Likelihood: Low

Description

The ERC7984TransferBatcher contract includes a retry mechanism that allows users to link a new transfer to a previously failed one. When `retryFor != 0`, an event is emitted linking the original transaction to the retry:

```
if (transfers[i].retryFor != 0) {
    emit RetryTransfer(address(cToken), msg.sender, transfers[i].
        retryFor, currentTxId);
}
```

The core purpose of this FHE-based token system is to keep transfer amounts confidential. Transfers that fail due to insufficient balance silently transfer zero to preserve privacy - otherwise, an observer could deduce balance information from reverts. However the retry mechanism does reference the original transfers/TXs being retried. If used as intended, that leaks some info which could be used to estimate the balances or amounts.

Let's say Alice enters with 100 cUSDC. She calls batcher to do 3 transfers, followed by the retry of the last transfer. Observer looking at the emitted events can assume that Alice probably ran out of funds to make 3rd transfer successfully, and can conclude with reasonable degree of certainty:

t1 + t2 + t3 > 100 t1 + t2 <= 100

This is a minor leak, as users can also send and receive funds outside of batcher, and even use batcher retry mechanism in non-standard ways (retry successful TX or retry failed one but with different amount/destination). But still it does allow for some level of statistical analysis which can lead to unwanted leaks to outside observers.

Recommendation

The existing batcher retry mechanism provides no real on-chain utility. Re-evaluate if it is really beneficial to have it place.

Client

Acknowledged, this design is for UX purposes.

BurraSec

Acknowledged.

[I-4] Minor code improvements

Target

- swap_v0.sol

- DeploymentCoordinator.sol
- ERC7984Upgradeable.sol

Severity

INFO

Description

- Storage mapping `_requestHandles` in `ERC7984Upgradeable` is unused
- Storage mapping `_allowances` in `RegulatedERC7984Storage` is unused
- `ERC7984StorageLocation` storage slot does not match the preimage
- When deploying, error `InsufficientDeployFee` can be misleading as fee can be overpaid, `IncorrectDeployFee` would be more precise
- No event emitted if `RegulatedERC7984Upgradeable` implementation is deployed during deploy
- `SENDER_NOT_A_COORDINATOR_WRAPPER` and `CALLED_BY_UNAUTHORIZED_WRAPPER` error codes are unused in `SwapV0`
- `feeAmount256 = 0;` set twice in `_finalizeSuccessfulUnwrap`
- There is no getter for `_receivers` in `WrapperUpgradeable`
- When paying the deployment fee in coordinator, the check `feeRecipient != address(0)` is unnecessary because of constructor and `setFeeRecipient` checks
- `SwapV0` should not inherit from ZamaEthereumConfig because it doesn't use any FHE operations

Recommendation

Fix the above items to have cleaner code

Client

Fixed in e940717f9f268abdf0020612b5e47db1baa74408.

BurraSec

Fix verified.